Computer Science 1
Guild Team Programming

Semester Project

Instructor: William A Loring

Table of Contents

Agile Software Development 4
- What is Agile? 4
- DRY Software Engineering 4
- SOLID Software Engineering 5

Guild Team Based Software Engineering 5
- Share Your Experience 6
- Guild Assignments 6
- Blackboard Groups 6
- Outlook Groups 6
- GitHub 7

Guild Software Engineering Process 7
  - Get Started 7
  - Development 7
  - Testing 7
  - Conclusion 8
- Sample KanBan Board 8
- Assignment Minimum Requirements 8

Guild Shared Software Engineering GitHub Process 8
- Organize 8
- Commits 9
- Before You Commit 9
- When You Commit 9

Example Project Workflow 9
- First 10
- Workflow 10

Week 7 Milestone: Guild Project Kickoff and Team Charter 10
- Assignment Submission 12

Week 8 Milestone: Design 12
- POS (Point of Sale) 13

GUI Design for Python 13
Assignment Submission 15
Week 10 Milestone 16
Requirements 16
2-Table ERD with Bridge Entity 16
Business Rules 17
Assignment Submission 18
Week 11 Milestone 19
Requirements 19
Shared Coding Process 19
Requirements 19
Assignment Submission 19
Week 12 Milestone 20
Requirements 20
Assignment Submission 20
Week 13 Milestone 21
Requirements 21
Assignment Submission 21
Week 14 Milestone 21
Requirements 21
Assignment Submission 21
Week 15 Milestone: 22
Requirements 22
Presentation Requirements 22
Assignment Submission 22
Week 16 Milestone 22
Program Requirements 22
Assignment Submission 23
Finals Week 23
Requirements 23
Assignment Submission 23

Agile Software Development

Agile development is one of the current processes for software development and other development activities.

What is Agile?

Agile software development refers to a group of software development methodologies based on iterative development, where requirements and solutions evolve through collaboration between self-organizing cross-functional teams. Agile methods or Agile processes generally promote a disciplined project management process that encourages frequent inspection and adaptation, a leadership philosophy that encourages teamwork, self-organization and accountability, a set of engineering best practices intended to allow for rapid delivery of high-quality software, and a business approach that aligns development with customer needs and company goals.

https://www.cprime.com/resources/what-is-agile-what-is-scrum

DRY Software Engineering

Don’t **R**epeat **Y**ourself (**DRY**) is a principle of software engineering aimed at reducing repetition of software patterns. It you are repeating any code, there is probably a better solution.

The DRY principle is stated as "Every piece of knowledge must have a single, unambiguous, authoritative representation within a system". This means that there shouldn’t be anything in your code that is duplicated somewhere else.

Violations of DRY are typically referred to as WET solutions, which is commonly taken to stand for "write every time", "write everything twice", "we enjoy typing" or "waste everyone's time".

Create classes with a single purpose or theme. Abstract as much as possible from the application to the classes. The application contains as little logic as possible. It creates and uses objects and their methods.

SOLID Software Engineering

In object-oriented computer programming, **SOLID** is a mnemonic acronym for five design principles intended to make software designs more understandable, flexible, and maintainable. The principles are a subset of many principles promoted by American software engineer and instructor Robert C. Martin, first introduced in his 2000 paper Design Principles and Design Patterns.

The **SOLID** concepts are:

- **The Single-responsibility principle:** a class should only have a single responsibility, that is, only changes to one part of the software's specification should be able to affect the specification of the class. A class should only have one job.
- **The Open–closed principle:** Classes should be open for extension and closed for modification. A class should be extendable without modifying the class itself. The code should not have to be changed, but new functionality can be added.
- **The Liskov substitution principle:** Objects in a program should be replaceable with instances of their subtypes without altering the correctness of that program. This means that, given that class B is a subclass of class A, we should be able to pass an object of class B to any method that expects an object of class A and the method should not give any weird output in that case. This is the expected behavior, because when we use inheritance, we assume that the child class inherits everything that the superclass has. The child class extends the behavior but never narrows it down.
- **The Interface segregation principle:** Many client-specific interfaces are better than one general-purpose interface. Clients should not be forced to implement a function they do not need.
- **The Dependency inversion principle:** Our classes should depend upon interfaces or abstract classes instead of concrete classes and functions.

Guild Team Based Software Engineering

Software engineering is rarely done alone. It is almost always done as part of a team.

The class will be divided into guilds of 2-3 people each. Guild membership is assigned by the Game Master (the instructor).

Each guild should come up with their own name (let's keep these PG-Rated please). Please let me know what your Guild name and number is. I will re name your GitHub repository.

Share Your Experience

In any class, some students, depending upon their major, programming experience, artistic talent, etc. may exhibit more proficiency than others on certain aspects of the assigned course work. A Guild gives an opportunity for everyone on the team to work together and share their experience.

This project is as much about working as a team as it is about the assignment. The process of coming together, helping each other out is a huge part of team-based software engineering.

Guild Assignments

Guild assignments are team projects where everyone works on the same code project.

Guild members are encouraged to be resources for the other Guild members. They can lend a hand if someone gets stuck on a problem. A 2nd eye on the code can find a missing semi colon or other minor error.

Individual assignments are not team projects. Everyone does their own work.

Blackboard Groups

Each Guild Team has a Group area in Blackboard under Guilds. This is used primarily used grading purposes.

Outlook Groups

An Outlook Group will be created for each Guild. This is used to share documents, send email, and work collaboratively.

Using Outlook Group Email and Files is a requirement of this project. How professionally, how responsive, and how often you communicate is part of your individual evaluation throughout the semester project.

Please address all Semester Project asynchronous communication to the Outlook Group Email address shown in your Outlook Group. There is no need to include separate email addresses. Everyone's wncc.edu email address is already in the Group Email address.

Each Team has a similar Outlook Group Email address. The last number is the only thing that is different.

For example, if you are in Group 1: Loring-Java-Group01

Using this address archives a copy of all email in the Groups feature in Outlook. You should see this group under Groups in Outlook. You can create new email messages from the Group.

Please watch this video on how to use Outlook Groups. https://youtu.be/DDG5n0QdvUg

How to Work with Word in Outlook Groups

GitHub

Each Guild will have a separate shared GitHub Repository. This repository should be used to store any code or text documentation pertinent to that assignment.

Guild Software Engineering Process

This is a highly suggested process that is like working in real world Agile software engineering.

Get Started

- Live scrum (meeting) to create pseudocode.
    - The pseudocode is a text file.
- Divide up coding into small tasks. Put in the KanBan board.
    - Read **GitHub KanBan Board** in Resources in Blackboard.

Development

- Comment your code descriptively.
    - Why did you do this, is there something left to do, did you hard code something for testing,
- Commit small changes early and often.
- Leave detailed commit descriptions.
    - Make sure the code compiles before pushing to GitHub.

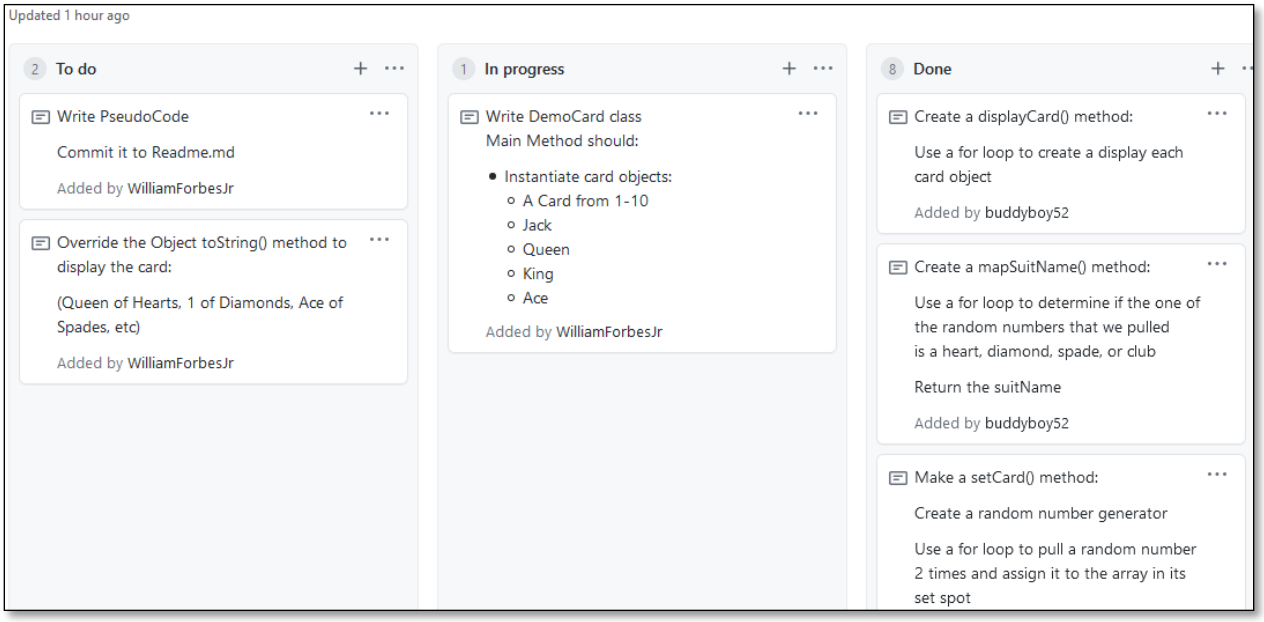
Testing

- Test the code to ensure it meets the requirements.

Conclusion

- Briefly describe who did what.
- Briefly describe your Guild's process.

Sample KanBan Board



Assignment Minimum Requirements

Think of this document as setting out the minimum requirements for the project. As long as the minimum requirements are met in your assignments, you have license to be creative and go further than the assignments request.

Guild Shared Software Engineering GitHub Process

Many of you are brand new to GitHub. We are going to use a basic approach. It requires communication with the team (which isn't a bad thing!).

Organize

- A divide-and-conquer approach is a good way to work on these assignments. When pair programming and working asynchronously, the number 1 thing is to organize the work first so people aren't writing duplicate methods. Even if you are perfectly committing, pushing, and pulling, you'll end up tripping over each other's code.
- Pens shouldn't touch paper until everyone knows exactly who is doing what.

Commits

- Commits should be small and frequent. Add a new method? Commit. Add a new data type? Commit. Have comments to add? Commit.
- Your commit messages should be written to not just say what you did, but why you took an approach. Treat this as your voice to the team, so they almost don't need to talk to you to see why you did something.
- The origin (GitHub) should be the source of truth for your work. If your local work is different, it needs to be committed or overridden with the origin since it's out of date.

Before You Commit

- **Pull**. You want to be sure that you're committing to the most recent version of the branch. Pulling right before committing will help avoid potential merge conflicts.

When You Commit

- **Push** immediately. That ensures that the origin (GitHub) is perfectly in sync with the most recent changes for everyone else. Remember: your commits are for everyone else so they're up to date with the current code.
- Message your group and let them know that you pushed. Yes, we should be pulling before committing, but it's a courtesy and avoids any potential issues.
- If you pull and there are conflicts that need to be resolved. . .
    - Say you pull and your teammate made some changes on that file. The easiest way to resolve this is to first copy your changes somewhere else (these should be small if we're committing correctly)
    - Accept the changes from the pull (Remember GitHub is the source of truth)
    - Add your changes back in
    - Commit then push.

Example Project Workflow

Here is an example of project workflow. You and I are on the same team on this project.

First

- We meet, talk over the pseudo, and setup a plan. You take conversion methods and I take display methods or something.
- Either setup the main class together or whoever can do it right away volunteers. We set it the main class together and push it.

Workflow

- I open GitHub before my IDE to check for updates. No updates.
- I pull and double-check I'm up to date.
- I write a small method displayInches()
- I pull again to check for changes
- Commit my changes. My commit comment says:
    - "Add displayInches method. Right now, it's hard-coded since we don't have a conversion yet but will update once we get data".
- Push
- I send message to the team:
    - Hi everyone I just pushed to add displayInches"
- Repeat

Your combined code is going to have different styles and approaches. That's perfectly okay and expected. When you go and work in the field, you will see a multitude of different styles and approaches in large codebases and will have to get used to that. If someone doesn't agree with a solution or approach, discuss it, and go with what you collectively decide.

Approach the project with the mindset that it is a single application and not a school assignment: 1 project 1 team. These discussions can be one of the bigger challenges of development and something you will regularly do day-to-day.

Week 7 Milestone: Guild Project Kickoff and Team Charter

100 points

Time required: 90 minutes

1. **Read:** Creating Productive Teams

2. **Read:** Elements of a High Performing Team
3. **Read:** Team Charter

Do: Determine how your Guild will handle the following items.

1. What is the name of your Guild?
2. Communication
    a. Synchronous (required)
        i. Skype
        ii. Zoom
        iii. Video conference software of your choice
    b. Asynchronous
        i. Outlook Group Email
        ii. Discord (Please send the instructor an invite if you choose Discord.)
3. Collaboration on documents
    a. Outlook Group Files (Required)
4. The Guild will create a list of the skills needed for the project.
    a. Team skills
    b. Computer skills
5. Each member will create a list of the skills they bring to the project.
    a. Team skills
    b. Computer skills
6. Guild leader of the week
    a. The Guild leader is not responsible for doing all the work, only organizing, or getting everyone together.
    b. Rotate between each member for Guild leader of the week.
    c. A recommended practice is to set a schedule for Guild leader rotation at the beginning of the project.

   d. The Guild leader is responsible for submitting the project assignment for that week.

7. Expectations for Guild Members
   a. Set your shared expectations for the Guild
   b. Responsiveness
      i. What are the expectations for response to communication?
   c. This is a collaborative team project.
      i. It is not ok to say to the rest of the team: plan this week without me and let me know what I am supposed to do.
      ii. Collaboration means each team member contributes equally.
   d. Deadlines for individual assignments
   e. Deadline for team assignments
   f. If these expectations are not met, ask the team member why
   g. If there is no response, move on without them
   h. Give an honest evaluation of individual performance with the evaluation

---

Assignment Submission

1. **Guild Team Charter:** The Guild leader submits the completed Guild Team Charter in Blackboard.
2. **Guild Individual Evaluation:** Each Guild member submits a Guild Individual Evaluation in Blackboard.

Week 8 Milestone: Design

100 points

Time Required: 90 minutes

We are going to use Agile development for this project. Please create a new folder for each week's work. Each week will have a functional deliverable.

This is an open-ended project. There are only a few requirements set out in this project document. At this stage of your learning, you have all the skills and experience necessary to design and create a project of your own design.

Python: OOP GUI

Java: OOP CLI or GUI

C++: OOP CLI

POS (Point of Sale)

We are going to create a POS (Point of Sale) system for a small business. You choose the business.

You can build from your program from last semester.

Theodora wants to track the following items. Start with the simple items, add what you can each week. You do not have to implement all these ideas.

- Persistent data stored on the hard drive
- Customer information
- Menu of items for sale
- Cash balance: Uncle Fred sold his ukulele for $500. We can use that for purchasing our initial inventory.
- Running inventory of items as they are sold
- Increase inventory when purchased from vendors
- Profit and Loss report
- Random disasters and windfalls
- A back story about the business

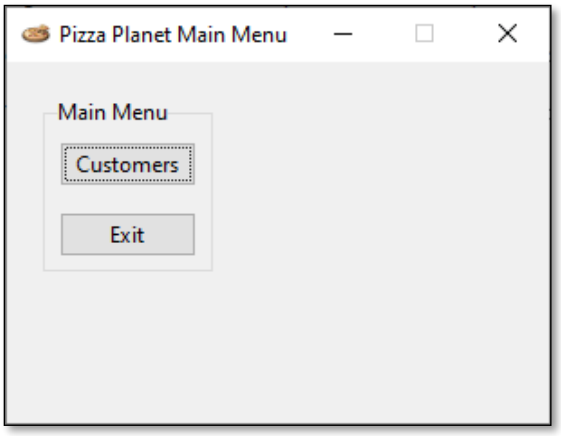
GUI Design for Python

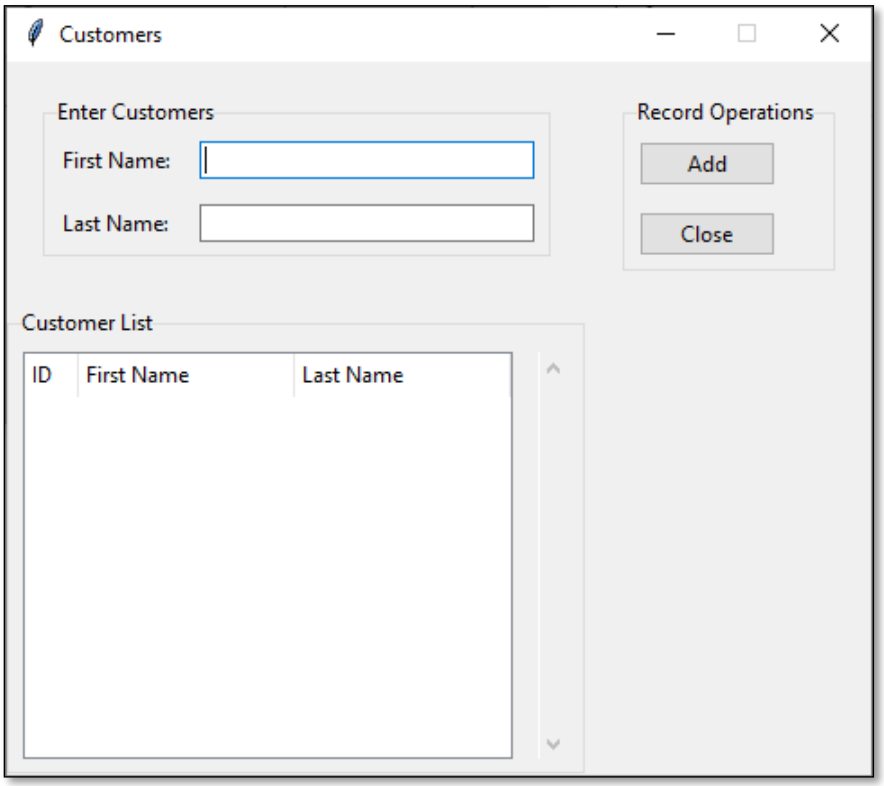
Draw the GUI on paper. We may want several screens, we have too much information for one screen. Each entity may need its own separate screen to modify its information.

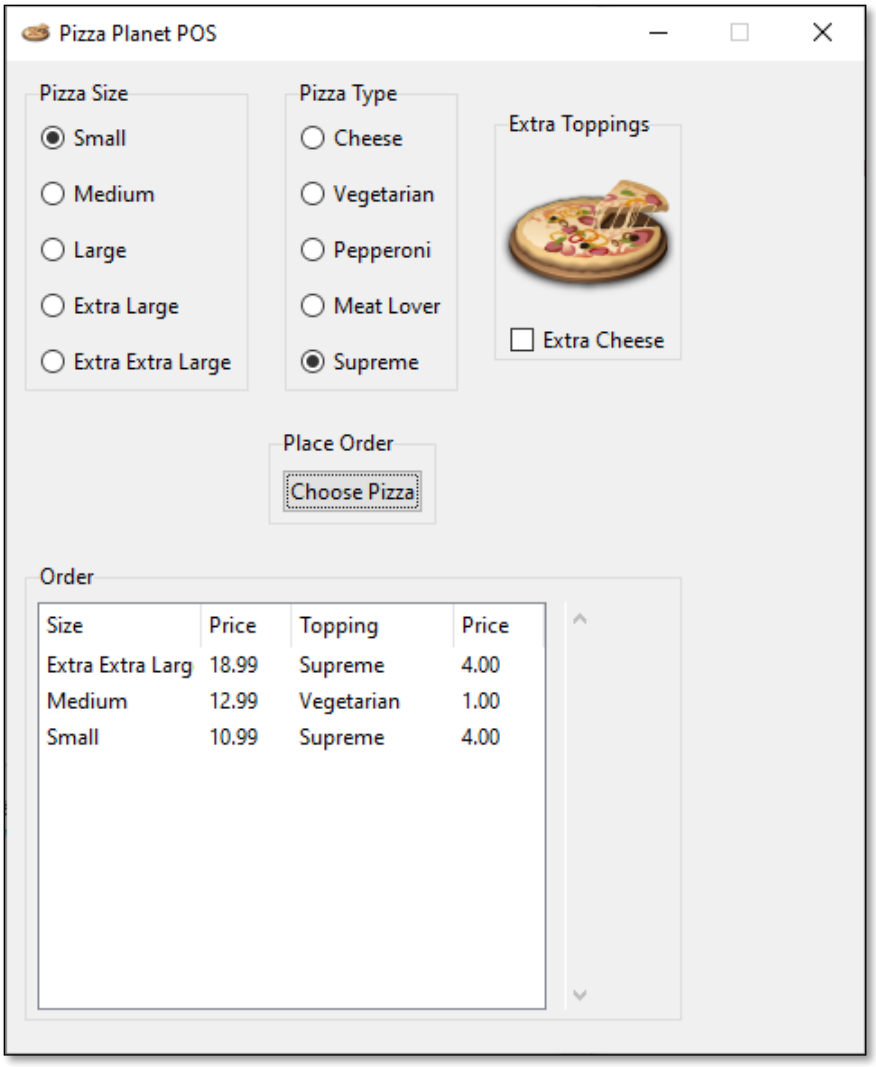
- Main screen
- Customer Entry

- Sale Entry
- Inventory Entry

Example run:







Assignment Submission

There is a link for each team.

https://classroom.github.com/a/zIQXnTf8

This is design week. What are you inspired to create?

Project Plan

1. Submit a project plan in Word for your development process and result.
2. Include requirements for your program.
3. Use TODO to sketch out your first iteration of your program.
4. Include a drawing of the interfaces.

1. **Milestone Folder:** Create a Milestone folder for each week's successful iteration of your project. Milestone 8, Milestone 9, etc. That will give you a way to easily go back to a known working version is the next version has trouble.
2. **Guild GitHub Assignment:** The Guild pseudocode, KanBan board, and code is created in GitHub.
3. **Guild Team Submission in Blackboard:** The Guild leader submits a screenshot of their GitHub repository.
4. **Guild Individual Evaluation:** Each Guild member submits an Individual Guild Evaluation in Blackboard.

Week 10 Milestone

100 points

Time Required: 90 minutes

Requirements

Create a working prototype draft initial coding of your project.

Create an ERD of your database. Start with the following tables.

1. Customers
2. Products
3. Transactions bridge table (sales)

2-Table ERD with Bridge Entity

This is where database planning starts.

We have our entities. Like OOP, entities represent something in the real world we want to keep track of.

- Customer
- Product

Business Rules

Business rules are how the entities interact. A functional real-life customer sales tracking database would have these business rules.

- A product can be sold to many customers.
- A customer can purchase many products.

This is an example of many to many relationships. You can't have many to many relationships in SQL. You can have two tables with a bridge or junction table connecting them as shown below to implement the many to many business rules.

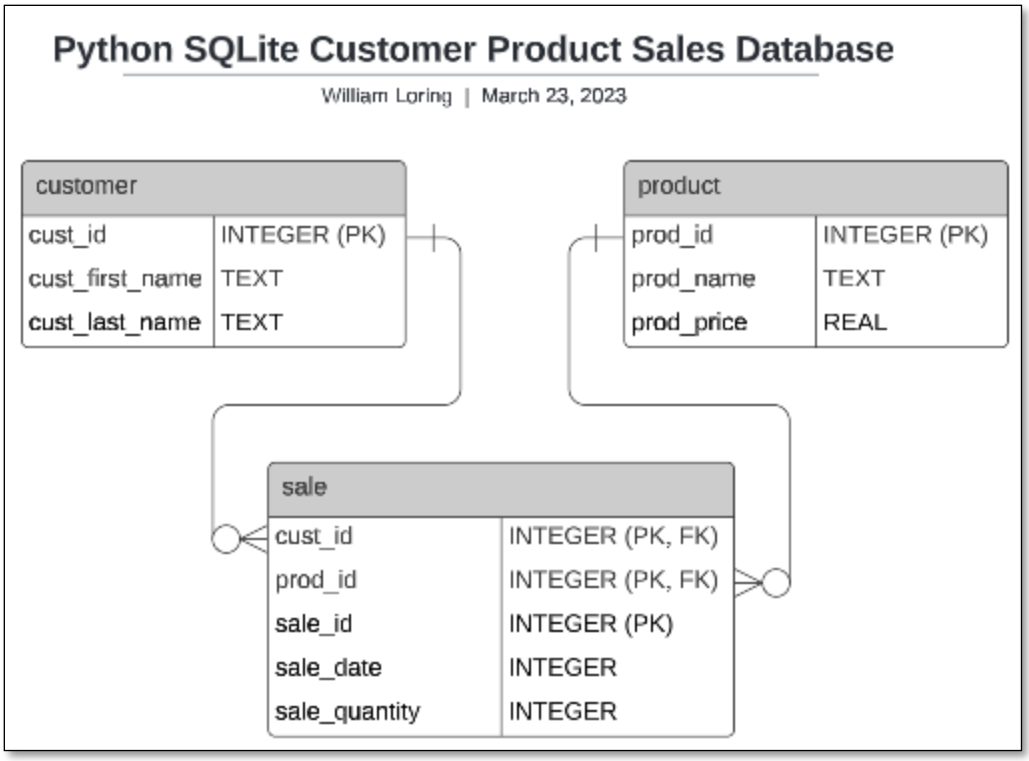
Primary Key: A primary key is a column or a set of columns in a table whose values uniquely identify a row in the table.

Foreign Key: A foreign key is a column or a set of columns in a table whose values correspond to the values of the primary key in another table.

Composite Key: A composite key is made by the combination of two or more columns in a table that can be used to uniquely identify each row in the table when the columns are combined uniqueness of a row is guaranteed, but when it is taken individually it does not guarantee uniqueness, or it can also be understood as a primary key made by the combination of two or more attributes to uniquely identify every row in a table.

These two tables are related through a composite primary key in the sale table. This composite primary key connects the two tables.

This is an example to get you started. You will want to eventually add more fields.



Assignment Submission

There is a repository for each team.

https://classroom.github.com/a/zIQXnTf8

We are using Agile development. We want a functioning deliverable each week.

1. **Milestone Folder:** Create a Milestone folder for each week's successful iteration of your project. Milestone 8, Milestone 9, etc. That will give you a way to easily go back to a known working version if the next version has trouble.
2. **Guild GitHub Assignment:** The Guild pseudocode, KanBan board, and code is created in GitHub.
3. **Guild Team Submission in Blackboard:** The Guild leader submits a screenshot of their GitHub repository.
4. **Guild Individual Evaluation:** Each Guild member submits an Individual Guild Evaluation in Blackboard.

Week 11 Milestone

Requirements

- Comment each line of code as shown in the tutorials and other code examples.
- Follow all directions carefully and accurately.
- Think of the directions as minimum requirements.

Shared Coding Process

1. Pseudocode first
2. Split up coding tasks
3. KanBan
4. Code and communicate
5. Commit often
6. Test and submit

Requirements

It is up to you to add your planned features to the program.

Please provide evidence of planning, teamwork, and collaboration.

One of the feature requirements is persistent data storage.

SQLite

SQLite allows for more robust data storage. This is one of the most popular methods of storing data. SQL databases are everywhere! You retrieved this assignment from an SQL database. Blackboard uses Java Server Pages and MySQL. WNCC uses Microsoft Active Server Pages and Microsoft SQL.

Assignment Submission

There is a link for each team.

https://classroom.github.com/a/zIQXnTf8

1. **Milestone Folder:** Create a Milestone folder for each week's successful iteration of your project. Milestone 8, Milestone 9, etc. That will give you a way to easily go back to a known working version if the next version has trouble.
2. **Guild GitHub Assignment:** The Guild pseudocode, KanBan board, and code is created in GitHub.
3. **Guild Team Submission in Blackboard:** The Guild leader submits a screenshot of their GitHub repository.
4. **Guild Individual Evaluation:** Each Guild member submits an Individual Guild Evaluation in Blackboard.

Week 12 Milestone

This project uses the same GitHub assignment repository as the last assignment.

Requirements

Next iteration with a working deliverable.

Assignment Submission

There is a link for each team.

https://classroom.github.com/a/zIQXnTf8

1. **Milestone Folder:** Create a Milestone folder for each week's successful iteration of your project. Milestone 8, Milestone 9, etc. That will give you a way to easily go back to a known working version if the next version has trouble.
2. **Guild GitHub Assignment:** The Guild pseudocode, KanBan board, and code is created in GitHub.
3. **Guild Team Submission in Blackboard:** The Guild leader submits a screenshot of their GitHub repository.
4. **Guild Individual Evaluation:** Each Guild member submits an Individual Guild Evaluation in Blackboard.

Week 13 Milestone

Requirements

Next iteration with a working deliverable.

Assignment Submission

There is a link for each team.

https://classroom.github.com/a/zIQXnTf8

1. **Milestone Folder:** Create a Milestone folder for each week's successful iteration of your project. Milestone 8, Milestone 9, etc. That will give you a way to easily go back to a known working version if the next version has trouble.
2. **Guild GitHub Assignment:** The Guild pseudocode, KanBan board, and code is created in GitHub.
3. **Guild Team Submission in Blackboard:** The Guild leader submits a screenshot of their GitHub repository.
4. **Guild Individual Evaluation:** Each Guild member submits an Individual Guild Evaluation in Blackboard.

Week 14 Milestone

Requirements

Next iteration with a working deliverable.

Assignment Submission

There is a link for each team.

https://classroom.github.com/a/zIQXnTf8

1. **Milestone Folder:** Create a Milestone folder for each week's successful iteration of your project. Milestone 8, Milestone 9, etc. That will give you a way to easily go back to a known working version is the next version has trouble.
2. **Guild GitHub Assignment:** The Guild pseudocode, KanBan board, and code is created in GitHub.

3. **Guild Team Submission in Blackboard:** The Guild leader submits a screenshot of their GitHub repository.
4. **Guild Individual Evaluation:** Each Guild member submits an Individual Guild Evaluation in Blackboard.

Week 15 Milestone:

Please read all the directions before beginning the assignment.

Requirements

Next iteration with a working deliverable.

Presentation Requirements

This will be a screencast with each member presenting a part of the program.

1. Describe how you developed your program.
2. Demonstrate the flow of the program.
3. Lessons learned.

Assignment Submission

1. **Milestone Folder:** Create a Milestone folder for each week's successful iteration of your project. Milestone 8, Milestone 9, etc. That will give you a way to easily go back to a known working version if the next version has trouble.
2. **Guild GitHub Assignment:** The Guild pseudocode, KanBan board, and code is created in GitHub.
3. **Guild Team Submission in Blackboard:** The Guild leader submits a screenshot of their GitHub repository.
4. **Guild Individual Evaluation:** Each Guild member submits an Individual Guild Evaluation in Blackboard.

Week 16 Milestone

Program Requirements

Next iteration with a working deliverable.

- Polish your program. Make it look nice.
- Test everything.
- Comment the code so anyone can understand your program.
- Reorganize your code to optimize it.

Assignment Submission

There is a link for each team.

https://classroom.github.com/a/zIQXnTf8

1. **Milestone Folder:** Create a Milestone folder for each week's successful iteration of your project. Milestone 8, Milestone 9, etc. That will give you a way to easily go back to a known working version if the next version has trouble.
2. **Guild GitHub Assignment:** The Guild pseudocode, KanBan board, and code is created in GitHub.
3. **Guild Team Submission in Blackboard:** The Guild leader submits a screenshot of their GitHub repository.
4. **Guild Individual Evaluation:** Each Guild member submits an Individual Guild Evaluation in Blackboard.

Finals Week

Final submission of your application.

Requirements

A fully functioning, polished, full stack application.

Assignment Submission

https://classroom.github.com/a/zIQXnTf8

1. **Guild GitHub Assignment:** The Guild pseudocode, KanBan board, and code is stored in GitHub.
2. **Guild Team Submission in Blackboard:** The Guild leader submits a screenshot of their GitHub repository.

3. **Guild Individual Evaluation:** Each Guild member submits an Individual Guild Evaluation in Blackboard.